

# Developer Guide — Decoder

From first project to a fully featured DCC decoder

*Covers the library on any platform; the TI MSPM0G3507 LaunchPad is the worked example*

*Generated 2026-09-25 from the repository sources by documentation/guides/build\_pdfs.py*

# Table of Contents

|                                              |           |
|----------------------------------------------|-----------|
| <b>1. Introduction</b>                       | <b>3</b>  |
| 1.1 What the Library Does (and Does Not Do)  | 3         |
| 1.2 Platform Support                         | 3         |
| <b>2. DCC Decoder Concepts</b>               | <b>4</b>  |
| 2.1 Signal Reception                         | 4         |
| 2.2 Bit Classification                       | 4         |
| 2.3 Packet Structure                         | 4         |
| 2.4 Address Matching                         | 4         |
| 2.5 Configuration Variables                  | 5         |
| <b>3. Project File Structure</b>             | <b>6</b>  |
| <b>4. dcc_user_config.h in Depth</b>         | <b>7</b>  |
| <b>5. Initialization — decoder.c</b>         | <b>8</b>  |
| 5.1 The dcc_config_t Struct (Decoder Fields) | 8         |
| 5.2 Edge Timestamp Ring Buffer               | 9         |
| 5.3 Setup and Main Loop                      | 9         |
| <b>6. ISR Architecture</b>                   | <b>11</b> |
| 6.1 GPIO Edge Interrupt                      | 11        |
| 6.2 100 ms Tick                              | 11        |
| <b>7. Implementing the Drivers</b>           | <b>12</b> |
| <b>8. The Bit Decoder</b>                    | <b>13</b> |
| <b>9. The Packet Decoder</b>                 | <b>14</b> |
| <b>10. CV Storage, Lock, and CV 29</b>       | <b>15</b> |
| <b>11. Service Mode (Decoder Side)</b>       | <b>16</b> |
| <b>12. RailCom Replies</b>                   | <b>17</b> |
| <b>13. Fail-Safe</b>                         | <b>18</b> |
| <b>14. Porting to a New MCU</b>              | <b>19</b> |
| <b>15. Unit Testing</b>                      | <b>20</b> |
| <b>16. Troubleshooting</b>                   | <b>21</b> |

# 1. Introduction

OpenDccCLib is a C library that implements the NMRA DCC protocol for microcontrollers. This guide covers the decoder role: receiving DCC from the track, recognizing the commands addressed to this decoder, and acting on speed, function, accessory, and CV programming commands. It allocates no dynamic memory and needs no operating system; a main loop and one edge interrupt are enough.

## 1.1 What the Library Does (and Does Not Do)

The library classifies one and zero bits from edge timing, assembles packets and validates the XOR byte, matches locomotive addresses against the CVs (accessory commands are delivered unfiltered for you to match), dispatches commands to your callbacks from the main loop, wraps CV storage with the decoder lock and factory reset, answers service-mode programming with the 6 ms ACK pulse, runs the S-9.2.4 packet time-out, and, when wired, transmits RailCom replies during the cutout.

It does not include motor control, lighting, servos, sound, or persistent storage. You supply those through function pointers in one `dcc_config_t` struct, and the library calls them.

## 1.2 Platform Support

| Processor / Board       | Example IDE / Toolchain                            |
|-------------------------|----------------------------------------------------|
| TI MSPM0G3507 LaunchPad | Code Composer Studio / Theia (the shipped example) |
| RP2040 / RP2350         | Arduino IDE, PlatformIO, Pico SDK                  |
| STM32 (F4 and others)   | STM32CubeIDE                                       |
| ESP32                   | Arduino IDE, PlatformIO                            |
| Any ARM Cortex-M        | GCC + Makefile                                     |

*The only hardware a decoder needs is a GPIO edge interrupt and a microsecond timestamp. RailCom transmit additionally needs a delay accurate to 1  $\mu$ s, and the edge interrupt must then call the library directly.*

## 2. DCC Decoder Concepts

### 2.1 Signal Reception

The track carries a bipolar square wave. The decoder rectifies it for power and converts it to a logic-level signal through an optocoupler or divider. Every transition is a half-bit boundary; the decoder measures the time between transitions.

### 2.2 Bit Classification

| Interval between edges        | Classification                                             | Define (dcc_defines.h)                 |
|-------------------------------|------------------------------------------------------------|----------------------------------------|
| < 80 $\mu$ s                  | One-bit half (58 $\mu$ s nominal)                          | DCC_DECODER_HALF_BIT_THRESHOLD_US = 80 |
| 80 $\mu$ s ... 10 000 $\mu$ s | Zero-bit half ( $\geq 95$ $\mu$ s; NMRA allows stretching) |                                        |
| $\geq 10\,000$ $\mu$ s        | No signal; the decoder resets to preamble search           | DCC_DECODER_HALF_BIT_MAX_US = 10000    |

Two consecutive halves of the same class make one bit. A short half followed by a long one does not pair; the long half starts a new pair.

### 2.3 Packet Structure

A valid preamble is at least `DCC_PREAMBLE_BITS_DECODER_MIN` = 10 one-bits (command stations send 14 or more). A zero start bit begins each byte; a one end bit closes the packet. The last byte is the XOR of all preceding bytes; a mismatch discards the packet. Packets shorter than 2 bytes or longer than `DCC_PACKET_MAX_BYTES` = 6 are discarded, and with RailCom compiled in the end bit is not counted toward the next preamble.

### 2.4 Address Matching

The library reads its addressing CVs at `DccConfig_initialize()` and again whenever a DCC packet writes one of them (CV 1, 8, 17, 18, 29, 513, 521, 541). Writes made by the application through `cv_write` do not refresh the cache; the demo's ADDR command re-calls `DccConfig_initialize()` for that reason.

| CV          | Purpose                                                                                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CV 1        | Primary (short) address, 1–127                                                                                                                                              |
| CV 17–18    | Extended (long) address, 128–10239                                                                                                                                          |
| CV 29 bit 5 | 0 = use CV 1, 1 = use CV 17–18                                                                                                                                              |
| CV 19       | Consist address; not read by the library in this release, so consist-addressed packets are not matched                                                                      |
| CV 513, 521 | Accessory decoder address: low 6 bits and high 3 bits in decoder-address mode; in output-address mode (CV 541 bit 6) the flat address is $CV\ 513 + 256 \times CV\ 521 - 1$ |
| CV 541      | Accessory configuration: bit 7 accessory decoder, bit 6 output-address mode, bit 5 extended                                                                                 |

Broadcast (address 0) is always accepted for multifunction packets. Accessory packets are not filtered: every basic and extended accessory command on the track reaches the accessory callbacks, and the

application compares the address.

## 2.5 Configuration Variables

| CV      | Name                                            | Note                                                                                                                                                                                           |
|---------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | Primary address                                 | Demo default 3                                                                                                                                                                                 |
| 2–6     | Vstart, acceleration, deceleration, Vhigh, Vmid | Application use                                                                                                                                                                                |
| 7       | Manufacturer version                            | Read-only per S-9.2.2; the library does not enforce it, so refuse the write in <code>cv_write</code>                                                                                           |
| 8       | Manufacturer ID                                 | Read-only per S-9.2.2; <b>writing the value 8 triggers</b> <code>factory_reset</code> (bypassing the decoder lock); any other value is forwarded to <code>cv_write</code> , so refuse it there |
| 11      | Packet time-out                                 | Fail-safe, in 100 ms units; 0 disables                                                                                                                                                         |
| 15 / 16 | Decoder lock                                    | Writes are refused unless CV 15 equals CV 16                                                                                                                                                   |
| 17–18   | Extended address                                |                                                                                                                                                                                                |
| 19      | Consist address                                 |                                                                                                                                                                                                |
| 28      | RailCom configuration                           | bit 0 Channel 1, bit 1 Channel 2; not read by the library in this release, the transmit engine always sends Channel 1                                                                          |
| 29      | Configuration                                   | See <code>DCC_CV29_*_BIT</code> ; bit 6 is reserved and forced to 0                                                                                                                            |
| 31 / 32 | Indexed CV page                                 | Selects the page behind CVs 257–512                                                                                                                                                            |
| 257–512 | Indexed window                                  | Served by <code>cv_read_indexed</code> / <code>cv_write_indexed</code> when wired                                                                                                              |

### 3. Project File Structure

|                                               |                                                                |
|-----------------------------------------------|----------------------------------------------------------------|
| decoder/                                      | <- your project folder                                         |
| decoder.c                                     | <- main entry: config struct, edge ISR, ring buffer, main loop |
| dcc_user_config.h                             | <- REQUIRED: role flags and limits                             |
| decoder_command_parser.c/h                    | <- demo UART command-line interface                            |
| application_callbacks/                        |                                                                |
| callbacks_dcc.c/h                             | <- CV storage stub, CV29 feature mask, command callbacks       |
| application_drivers/                          |                                                                |
| ti_driverlib_dcc_driver.c/h                   | <- timestamp, lock, RailCom delay                              |
| ti_driverlib_uart_driver.c/h                  | <- UART I/O                                                    |
| ack_pulse_driver.c/h                          | <- ACK current load                                            |
| dcc_lib/ -> ../../../../src/dcc               | <- library core (symlink; do not edit)                         |
| dcc_config.h/c                                | - config struct, lifecycle, wiring of every module             |
| dcc_bit_decoder.h/c                           | - edge timing -> bits -> bytes                                 |
| dcc_packet_decoder.h/c                        | - XOR, address match, dispatch queue                           |
| dcc_cv_storage.h/c                            | - CV access, decoder lock, factory reset, indexed CVs          |
| dcc_failsafe.h/c                              | - CV 11 packet time-out                                        |
| dcc_railcom_decoder.h/c                       | - RailCom transmit engine (bit-bang)                           |
| dcc_railcom_utilities.h/c                     | - 4/8 code words                                               |
| dcc_application_decoder_cv.h/c                | - CV read/write API                                            |
| dcc_application_decoder_railcom.h/c           | - RailCom reply API                                            |
| dcc_application_accessory_decoder_railcom.h/c | - accessory-decoder RailCom API                                |

## 4. dcc\_user\_config.h in Depth

```
#define DCC_COMPILE_DECODER
#define DCC_COMPILE_RAILCOM // RailCom transmit; comment out to strip
// #define DCC_COMPILE_ACCESSORY_DECODER // accessory-decoder RailCom helpers; needs DCC_COMPILE_RA

#define USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS 29 // required by the build; not used by the library, F
#define USER_DEFINED_DCC_DECODER_PACKET_QUEUE_DEPTH 8 // deferred dispatch queue, >= 2
```

| Define                                      | Meaning                                                                                                                                                                                         |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DCC_COMPILE_DECODER                         | Bit decoder, packet decoder, CV storage, fail-safe                                                                                                                                              |
| DCC_COMPILE_RAILCOM                         | Compiles the RailCom transmit engine and its config fields (railcom_tx_pin_set, railcom_delay_us, on_railcom_request); active only when railcom_tx_pin_set is wired                             |
| DCC_COMPILE_ACCESSORY_DECODER               | Accessory-decoder RailCom reply helpers (SRQ, status, time, error); needs DCC_COMPILE_RAILCOM. Packet reception lives under DCC_COMPILE_DECODER, so an accessory-only build has no receive path |
| USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS      | Required by the build ( $\geq 1$ ) but not used by the library; F0–F68 are always dispatched                                                                                                    |
| USER_DEFINED_DCC_DECODER_PACKET_QUEUE_DEPTH | Packets that can wait for main-loop dispatch; one slot is reserved, so $\geq 2$ and the queue holds depth – 1. A full queue drops the newest packet                                             |

A decoder project must not define DCC\_COMPILE\_COMMAND\_STATION unless it really is both, as a booster or repeater would be.

## 5. Initialization — decoder.c

### 5.1 The dcc\_config\_t Struct (Decoder Fields)

| Field                                                              | Required?                                | Purpose                                                                                                                                                                                            |
|--------------------------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| lock_shared_resources, unlock_shared_resources, get_timestamp_usec | REQUIRED                                 | Platform: interrupt mask and microsecond clock                                                                                                                                                     |
| cv_read(cv, *value), cv_write(cv, value)                           | REQUIRED                                 | Persistent CV storage                                                                                                                                                                              |
| cv29_apply_supported_features(flags)                               | REQUIRED by contract (NULL is tolerated) | On every CV 29 write, clear the feature bits this product does not implement; the library stores what you leave set                                                                                |
| factory_reset()                                                    | optional                                 | Called when the value 8 is written to CV 8; bypasses the decoder lock                                                                                                                              |
| cv_read_indexed, cv_write_indexed                                  | optional                                 | Serve CVs 257–512 for the page in CV 31:32                                                                                                                                                         |
| railcom_tx_pin_set(high)                                           | optional (DCC_COMPILE_RAILCOM only)      | Drive the RailCom current source. NULL = no RailCom transmit                                                                                                                                       |
| railcom_delay_us(us)                                               | required with the above                  | Busy-wait accurate to 1 µs; the library calls it with 4 (bit), 33 (gap) and 80 (blanking) µs                                                                                                       |
| on_railcom_request(instruction, count, *out)                       | optional                                 | Fill a Channel 2 reply for a command addressed to this decoder; return DATA, ACK, BUSY, NACK or NONE. Called on the edge path, not from the main loop; keep it fast                                |
| start_ack_pulse(), stop_ack_pulse()                                | optional                                 | Service-mode ACK current load; the library times the 6 ms from DccConfig_run(). A second ACK while the pulse is active is ignored, not restarted; with start_ack_pulse NULL no pulse is ever armed |
| on_*_command(...), on_n_failsafe_entered/exited                    | optional                                 | Application notifications, main loop                                                                                                                                                               |

```
const dcc_config_t dcc_config = {
    .lock_shared_resources = &TI_DccDriver_lock_shared_resources,
    .unlock_shared_resources = &TI_DccDriver_unlock_shared_resources,
    .get_timestamp_usec = &TI_DccDriver_get_timestamp_usec,

    .cv_read = &CallbacksDcc_cv_read, /* RAM stub in the demo */
    .cv_write = &CallbacksDcc_cv_write,
    .factory_reset = &CallbacksDcc_factory_reset,
    .cv_read_indexed = &CallbacksDcc_cv_read_indexed,
    .cv_write_indexed = &CallbacksDcc_cv_write_indexed,
    .cv29_apply_supported_features = &CallbacksDcc_cv29_apply_supported_features,

    .railcom_tx_pin_set = NULL, /* no current source on the demo board */
    .railcom_delay_us = &TI_DccDriver_railcom_delay_us, /* uses DELAY_TIMER_INST, which must be added in Sys

    .start_ack_pulse = &AckPulseDriver_start,
    .stop_ack_pulse = &AckPulseDriver_stop,

    .on_speed_command = &CallbacksDcc_on_speed_command,
```



```

/* ... the other command callbacks ... */
.on_failsafe_entered = &CallbacksDcc_on_failsafe_entered,
.on_failsafe_exited  = &CallbacksDcc_on_failsafe_exited,
};

```

## 5.2 Edge Timestamp Ring Buffer

The GPIO interrupt only captures a timestamp into a power-of-two ring; the main loop drains it into `DccConfig_decoder_edge_isr()`. That keeps the interrupt under a microsecond so it never overruns the 58  $\mu$ s half-bit, and it means bit decoding runs in the main loop on the demo. Calling `DccConfig_decoder_edge_isr()` directly from the interrupt is equally valid on a faster part, and it is required once RailCom transmit is wired: the reply is timed from the end-bit edge, and a main-loop drain delays it past the cutout.

```

#define EDGE_BUF_SIZE 256 /* power of two */
static volatile uint32_t _edge_buf[EDGE_BUF_SIZE];
static volatile uint16_t _edge_head, _edge_tail;

void GROUP1_IRQHandler(void) { /* PB1 main, PB4 service */
    bool track_sel = /* PB17 reads high */;
    if ((!track_sel && /* edge on PB1 */) || (track_sel && /* edge on PB4 */)) {
        uint16_t next = (_edge_head + 1) & (EDGE_BUF_SIZE - 1);
        if (next != _edge_tail) { /* ring full: the edge is dropped */
            _edge_buf[_edge_head] = TI_DccDriver_get_timestamp_usec();
            _edge_head = next;
        }
        /* toggle PB3 (logic-analyzer aid) */
    }
    /* clear the interrupt flags */
}

static void _drain_edge_buffer(void) {
    while (_edge_tail != _edge_head) {
        DccConfig_decoder_edge_isr(_edge_buf[_edge_tail]);
        _edge_tail = (_edge_tail + 1) & (EDGE_BUF_SIZE - 1);
    }
}

```

## 5.3 Setup and Main Loop

```

int main(void) {
    SYSCFG_DL_init();
    NVIC_EnableIRQ(GPIOB_INT_IRQn);
    NVIC_EnableIRQ(TIMESTAMP_TIMER_INST_INT_IRQn); /* timestamp overflow counter */
    NVIC_EnableIRQ(ACK_PULSE_TIMER_INST_INT_IRQn);
    TI_DccDriver_initialize();
    TI_UartDriver_initialize();
    AckPulseDriver_initialize();
    CallbacksDcc_initialize(); /* CV defaults: CV1 = 3, CV29 = 0x06 */
    DccConfig_initialize(&dcc_config); /* reads the addressing CVs */
    DecoderCommandParser_initialize();

    while (1) {
        _drain_edge_buffer(); /* edges -> bit decoder */
        DccConfig_run(); /* packet dispatch, callbacks, ACK timing, fail-safe */
        CallbacksDcc_drain(); /* RECV lines -> UART */
        TI_UartDriver_echo_process();
        DecoderCommandParser_process();
    }
}

```

*If the ring buffer overflows, edges are dropped silently and packets are lost. Keep every call in the loop non-blocking.*

## 6. ISR Architecture

### 6.1 GPIO Edge Interrupt

Fires on every rising and falling edge of the DCC input. The demo has two inputs on one port, PB1 for the main track and PB4 for the service track, and a track-select input on PB17 chooses which one is accepted. During a RailCom transmit the library masks this interrupt through `lock_shared_resources`, so the decoder's own current pulses cannot trigger it.

### 6.2 100 ms Tick

The library's `DccConfig_100ms_timer_tick()` belongs to the command-station role; the decoder needs no periodic tick. The demo's SysTick only blinks the heartbeat LED. Fail-safe timing uses `get_timestamp_usec` from `DccConfig_run()`.

## 7. Implementing the Drivers

| Function                                        | Context                                                       | Contract                                                                                                                                                   |
|-------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| lock_shared_resources / unlock_shared_resources | any                                                           | Mask and restore interrupts; must cover the DCC edge interrupt                                                                                             |
| get_timestamp_usec                              | any                                                           | Free-running microsecond counter; monotonic, wrap is fine. Called on the edge path and from <code>DccConfig_run()</code> (fail-safe and ACK timing)        |
| cv_read / cv_write                              | main loop                                                     | Persistent storage. The demo uses a RAM array that resets at power-up; use Flash or EEPROM in a product                                                    |
| start_ack_pulse / stop_ack_pulse                | main loop                                                     | Switch the ACK current load; the library calls stop after <code>DCC_ACK_PULSE_DURATION_US = 6000 µs</code> , and ignores a start while the pulse is active |
| railcom_tx_pin_set                              | edge path (ISR when the interrupt calls the library directly) | Set the RailCom line to a UART level, true = mark                                                                                                          |
| railcom_delay_us                                | edge path                                                     | Spin that many microseconds, accurate to 1 µs; called with 4, 33 and 80 (a hardware timer or DWT cycle counter, for example)                               |

## 8. The Bit Decoder

`DccConfig_decoder_edge_isr(timestamp)` computes the interval since the previous edge and classifies it as in section 2.2. Halves pair into bits; ten or more consecutive ones arm the preamble; a zero then starts byte assembly. The finished packet is handed to the packet decoder's queue through an interface pointer, never by direct call; with RailCom compiled in it first passes through the RailCom transmit hook, and each completed byte also goes to the RailCom recognizer so the reply can be prepared before the packet ends.

## 9. The Packet Decoder

Completed packets are queued (`USER_DEFINED_DCC_DECODER_PACKET_QUEUE_DEPTH`) and dispatched from `DccConfig_run()`. This deferral keeps long handlers such as CV writes out of the edge path, and it is why every command callback runs in main-loop context (`on_railcom_request` is the exception; it runs on the edge path). For each packet the decoder validates the XOR, matches the address, then parses the instruction and calls the matching callback. A multifunction packet addressed to this decoder, or broadcast, also re-arms the fail-safe timer; accessory, service-mode, idle and reset packets do not. The reply engine's address is updated whenever the address-CV cache is refreshed, not per packet. A full queue drops the newest packet.

| Instruction                                                   | Callback                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Speed<br>14/28/128                                            | <code>on_speed_command(address, speed, direction, mode)</code> ; mode is a <code>dcc_speed_mode_enum</code> . direction already has CV 29 bit 0 applied, speed is 0 (stop) or 2 and up (e-stop goes to the next row), and 14 versus 28 steps follows CV 29 bit 1                                                                                                                       |
| Emergency<br>stop                                             | <code>on_emergency_stop_command(address)</code>                                                                                                                                                                                                                                                                                                                                        |
| Functions<br>F0–F68                                           | <code>on_function_command(address, function_number, state)</code>                                                                                                                                                                                                                                                                                                                      |
| Basic /<br>extended<br>accessory                              | <code>on_accessory_basic_command(board, pair, activate)</code> ,<br><code>on_accessory_extended_command(address, aspect)</code> ; delivered for every accessory packet, no address filter. In output-address mode (CV 541 bit 6) board is the 11-bit output address and pair is the R bit                                                                                              |
| CV write /<br>verify / bit,<br>main track or<br>service track | <code>on_cv_write_command(cv, value, service_mode)</code> , <code>on_cv_verify_command(...)</code> ,<br><code>on_cv_bit_command(cv, bit, value, service_mode)</code> . An operations-mode verify only notifies (nothing is read or compared); a service-mode bit write fires both the write and the bit callback; accessory operations-mode CV access is not delivered in this release |
| Consist                                                       | <code>on_consist_command(address, consist_address, direction_normal)</code> ; notification only, CV 19 is not stored and consist addresses are not matched                                                                                                                                                                                                                             |
| Binary state<br>short / long                                  | <code>on_binary_state_short_command(address, state, active)</code> ,<br><code>on_binary_state_long_command(...)</code>                                                                                                                                                                                                                                                                 |
| Analog<br>function                                            | <code>on_analog_function_command(address, output, value)</code>                                                                                                                                                                                                                                                                                                                        |

## 10. CV Storage, Lock, and CV 29

All CV traffic goes through `dcc_cv_storage`, which wraps your `cv_read` / `cv_write`:

- **Decoder lock.** A write is refused unless CV 15 equals CV 16; CV 15 and CV 16 themselves are always writable. If CV 15 or CV 16 cannot be read the decoder is treated as unlocked.
- **Factory reset.** A write of the value 8 to CV 8 calls `factory_reset()` instead of storing, and bypasses the lock; any other value is forwarded to `cv_write`.
- **Indexed CVs.** CVs 257–512 are routed to `cv_read_indexed` / `cv_write_indexed` with the page from CV 31:32, when those hooks are wired.
- **CV 29 feature mask.** On every CV 29 write the library forces the reserved bit 6 clear, decodes the byte into a `dcc_cv29_flags_t` (direction reversed, 28/128 steps, analog conversion, RailCom, speed table, extended address, accessory), and calls `cv29_apply_supported_features()`. Clear the flags your product does not implement; per S-9.2.2 an unsupported feature bit must never be settable, and only the application knows what it supports.

The application API is `DccApplicationDecoderCv_read`, `_write` and `_is_locked`, wired by `DccConfig_initialize()` onto the storage module, so the decoder lock, the CV 29 filter and the CV 8 reset apply to application writes too. Two differences from a write that arrives by DCC packet: the application path refuses every write while the decoder is locked, including the CV 8 reset, and it does not refresh the packet decoder's address-CV cache.

## 11. Service Mode (Decoder Side)

On the programming track the library recognizes the service-mode sequences and handles them: a write stores through `cv_write` and answers with the ACK pulse; a verify compares through `cv_read` and answers only on a match; bit operations do the same at bit level. Your CV callbacks fire afterwards with `service_mode` true, so you can tell programming-track writes from operations-mode ones.

Service mode is entered after 3 consecutive reset packets and left on the next packet that is neither idle nor a service-mode packet. Direct-mode packets reach CVs 1–1024. The 3-byte register and paged forms map registers 1–8 straight onto CVs 1–8: the page register (6) is stored as CV 6 and the page is not applied, and register 5 lands on CV 5 rather than CV 29, so only CVs 1–4 are reached correctly through those modes.



## 12. RailCom Replies

With `railcom_tx_pin_set` and `railcom_delay_us` wired, the library transmits during the cutout that follows a packet addressed to this decoder, bit-banging 4/8-encoded bytes at 250 kbaud with `DCC_RAILCOM_TX_BIT_US = 4 µs` bits, after a `DCC_RAILCOM_TX_BLANK_US = 80 µs` blanking delay. Interrupts are masked for the transmit window through `lock_shared_resources`. The reply is timed from the end-bit edge, so the edge interrupt must call `DccConfig_decoder_edge_isr()` directly rather than through a main-loop drain.

- **Channel 1** carries the address broadcast, alternating ADR1 and ADR2, sent automatically after a multifunction command addressed to this decoder whose length the recognizer can size. Broadcast and accessory packets, decoder control, XPOM, time and date, and system time get no reply. CV 28 and CV 29 bit 3 are not consulted.
- **Channel 2** is yours: `on_railcom_request()` is called as soon as a command addressed to this decoder is recognized, before its XOR byte, and returns a reply status. Return `DCC_RAILCOM_REPLY_DATA` with a `dcc_railcom_response_t` (the id and the first byte share the first two code words, each further byte carries 6 bits, and the encoded reply is limited to six code words, so at most five data bytes), or ACK, BUSY, NACK, or NONE.
- The helper API `DccApplicationDecoderRailcom_send_*` (address feedback, POM response, dynamic data, track-search response, CV auto-transfer, ACK, NACK, raw) exists, but `DccConfig_initialize()` does not initialize it and the transmit engine has no sink for it in this release, so its calls return without transmitting; answer through `on_railcom_request` instead.
- The accessory helper module `DccApplicationAccessoryDecoderRailcom_*` (`_get_srq_state`, `_send_srq`, `_send_status`, `_send_status_extended`, `_send_status_4`, `_send_time_report`, `_send_error_report`, and the stop-command and cutout hooks) has the same status, and the transmit engine does not answer accessory packets.

*The shipped decoder board has no current-source circuit, so the demo leaves `railcom_tx_pin_set` NULL and no reply is transmitted. The engine and encoders are unit-tested; the on-track side is a known open item, and the compliance overview's RailCom entries predate the transmit engine. Known limitation: a reply carrying only Channel 2 data is read as Channel 1 by this library's own command station.*

## 13. Fail-Safe

S-9.2.4 requires a decoder to stop everything when no packet addressed to it arrives within its time-out. The library implements this in `dcc_failsafe`: CV 11 times `DCC_FAILSAFE_CV11_UNIT_US = 100 000 µs`, polled from `DccConfig_run()`, stamped at initialization and re-armed by every multifunction packet for this decoder or broadcast (accessory and service-mode packets do not count). Once tripped it stays tripped until such a packet arrives. CV 11 = 0 disables it. `on_failsafe_entered()` is where you stop the motor and drop the outputs; `on_failsafe_exited()` fires when an addressed packet returns. A command station built on this library paces its refresh so an idle locomotive may go up to 120 packet cycles without an addressed packet (about 0.8 s, 1.8 s with worst-case packets), so on such a layout CV 11 should be 0 or at least the station's `DCC_REFRESH_CV11_FLOOR`, 20 (2.0 s) by default.

## 14. Porting to a New MCU

1. Copy the decoder example project.
2. Rewrite `ti_driverlib_dcc_driver.c/h`: interrupt mask, microsecond timestamp, and a 1  $\mu$ s-accurate `railcom_delay_us` if you transmit RailCom.
3. Replace `ack_pulse_driver.c/h` with your current-load control, or leave both ACK hooks NULL.
4. Replace the CV stub with Flash or EEPROM access.
5. Wire the edge interrupt to capture timestamps, and the ring-buffer drain (or, with RailCom, the interrupt itself) to `DccConfig_decoder_edge_isr()`.
6. Build and test against a known-good command station; the RECV lines over UART show every decoded command.

## 15. Unit Testing

Decoder-role tests, GoogleTest with mocked drivers, run with the rest of the suite: `cd test && make`. At generation time the whole suite is 29 binaries, 1246 tests, 0 failures, with 99.7 % line coverage.

| Test file                                                                                                                              | What it tests                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dcc_bit_decoder_Test</code>                                                                                                      | Edge classification, thresholds, preamble, noise and time-out recovery                                                                                              |
| <code>dcc_packet_decoder_Test</code>                                                                                                   | XOR, address matching for every address type, instruction dispatch, queue                                                                                           |
| <code>dcc_cv_storage_Test</code> ,<br><code>dcc_application_decoder_cv_Test</code>                                                     | Lock, factory reset, indexed CVs, CV 29 feature mask                                                                                                                |
| <code>dcc_failsafe_Test</code>                                                                                                         | CV 11 time-out, enter and exit                                                                                                                                      |
| <code>dcc_config_Test</code>                                                                                                           | Wiring and lifecycle, edge dispatch on run, the 6 ms ACK pulse (auto-stop, restart ignored, NULL hooks), the CV application API routed through storage and the lock |
| <code>dcc_railcom_decoder_Test</code> ,<br><code>dcc_railcom_utilities_Test</code> , <code>dcc_application_decoder_railcom_Test</code> | Reply engine, 4/8 encoding against the spec table, reply builders                                                                                                   |
| <code>dcc_application_accessory_decoder_railcom_Test</code>                                                                            | Accessory SRQ, status, time and error replies                                                                                                                       |

## 16. Troubleshooting

| Symptom                                  | Likely cause and fix                                                                                                                                                                                                                                    |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No packets decoded                       | Input wiring, edge interrupt not enabled, or the ring buffer is not drained                                                                                                                                                                             |
| Packets decode but callbacks stay silent | Address mismatch; check CV 1 and CV 29 bit 5, or try the broadcast address                                                                                                                                                                              |
| Frequent XOR failures                    | Noisy input; clean the signal to logic levels and check the threshold circuit                                                                                                                                                                           |
| Command station reports NO ACK           | ACK current load not wired or too weak; verify a 6 ms pulse on the ACK pin                                                                                                                                                                              |
| Fail-safe trips at once                  | No DCC reaching the decoder, or CV 11 set very small (units of 100 ms; keep it 0 or at least 20 with a refresh-pacing command station)                                                                                                                  |
| Functions work but speed does not        | CV 29 bit 1 speed-step setting does not match the command station                                                                                                                                                                                       |
| Address lost after power cycle           | CV storage is RAM in the demo; use non-volatile storage                                                                                                                                                                                                 |
| No RailCom reply seen                    | <code>railcom_tx_pin_set</code> is NULL, <code>railcom_delay_us</code> is inaccurate, edges are drained from the main loop instead of the interrupt, the packet was not addressed to this decoder, or the instruction is one the recognizer cannot size |

Library source: [github.com/JimKueneman/OpenDccCLib](https://github.com/JimKueneman/OpenDccCLib). NMRA DCC standards: [www.nmra.org/dcc](http://www.nmra.org/dcc).